
layout: single title: "Piggydrive: Access Cloud Storage on Linux When OAuth Blocks You" category: [posts] tags: piggydrive cloud-storage linux mac bridge oauth automation cli tools date: 2026-05-03 10:00:00 excerpt: "University and enterprise cloud storage often blocks third-party clients at the authentication layer. Piggydrive sidesteps the problem: use a trusted machine as a bridge, access files over a private network. The result? Full cloud access from Linux without fighting OAuth restrictions." hidden: true sitemap: false ---

If you've ever tried to access enterprise or university cloud storage from Linux, you know the pain. Institutional authentication policies often block third-party clients — `rclosure`, native CLI tools, even WebDAV mounts all hit the same wall: authentication rejected. The organization doesn't trust them.

{: style="text-align: justify;"}

I needed cloud storage access from Linux for research projects, course materials, and collaboration work. Every standard CLI tool I tried failed at authentication. So I built **Piggydrive**: a workaround that doesn't fight the restrictions — it goes around them.

{: style="text-align: justify;"}

The Problem: Authentication is a Wall

Enterprise and university cloud providers design their authentication to block anything they don't explicitly approve. For many organizations, that means:

- `rclosure` : Fails with "application not permitted" errors
- **Native CLI tools**: Authentication token exchange rejected
- **WebDAV mounts**: Blocked at the policy level
- **Direct API calls**: Same authentication blocker

{: style="text-align: justify;"}

The issue isn't your code. It's the organization's policy. You can't fix it with better authentication logic.

```
{: style="text-align: justify;"}
```

The Solution: A Trusted Bridge

Piggydrive's core idea is simple: **use a machine that's already trusted.**

Many of us have a personal laptop (Mac or Windows) that runs the official cloud storage client — the organization trusts it completely. That machine syncs the entire cloud folder locally. All we need is a way to access those files from Linux without re-authenticating.

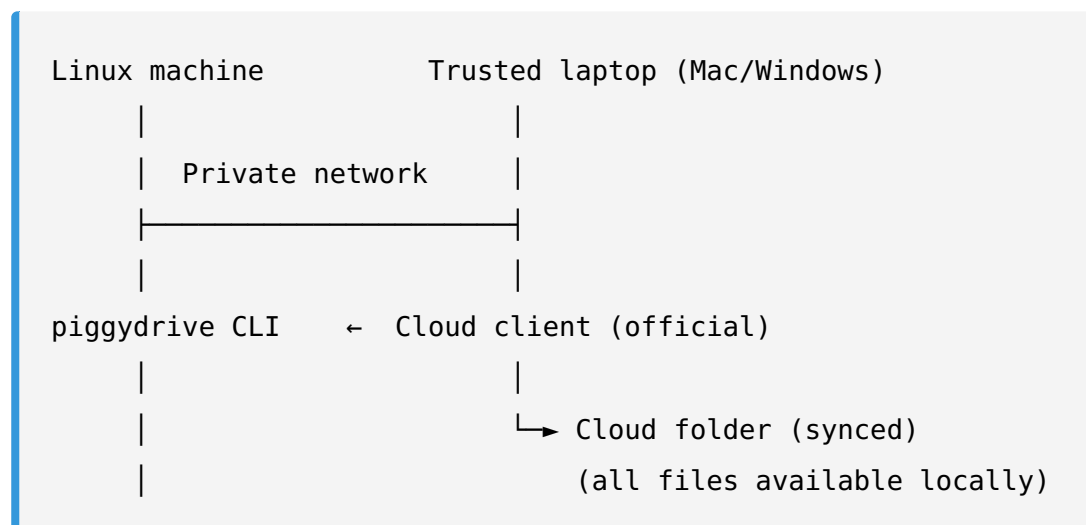
```
{: style="text-align: justify;"}
```

This is especially useful in university and corporate environments.

Institutions often provide you with a machine (laptop or desktop) that's already configured to access their cloud infrastructure. That machine is trusted by default. Instead of fighting to authenticate from Linux, use that trusted machine as a bridge.

```
{: style="text-align: justify;"}
```

The architecture:



```
{: style="text-align: justify;"}
```

The laptop runs a small daemon that exposes filesystem operations over a secure channel. The Linux `piggydrive` CLI sends commands, the laptop daemon executes them on the synced cloud folder, and returns results. No authentication dance on Linux — the laptop already authenticated.

{: style="text-align: justify;"}

How It Works (Technical Overview)

The Bridge

The laptop daemon is a Python server that:

1. **Listens on a private network** — secure connection between trusted devices
2. **Executes filesystem operations** — list, read, write, move, delete
3. **Handles cloud sync** — triggers download of cloud-only files when needed
4. **Returns structured data** — clean API for the CLI to parse

{: style="text-align: justify;"}

The key insight: I'm not building a cloud storage client. I'm building a **remote filesystem proxy**. The laptop's official cloud client does all the heavy lifting — authentication, sync, conflict resolution. The daemon just exposes the local folder.

{: style="text-align: justify;"}

The CLI

The Linux `piggydrive` CLI provides a clean interface:

```
# Search for files (fast, even on large folders)
piggydrive find "project" --max 50

# List a directory
```

```
piggydrive ls /Projects/research
```

```
# Pull a file to local disk
```

```
piggydrive pull "/Cloud/research/paper.pdf" ~/work/paper.pdf
```

```
# Push a file to cloud storage
```

```
piggydrive push ~/work/report.md "/Cloud/reports/report.md"
```

```
# Check sync status
```

```
piggydrive sync-status
```

{: style="text-align: justify;"}

Fast Search

The killer feature: `piggydrive find` uses the laptop's built-in search (Spotlight on Mac, Windows Search on PC) for instant results. Even on a 200,000+ file tree, results come back in under a second. It searches filenames, not contents, and works on cloud-only placeholders — no need to download files just to find them.

{: style="text-align: justify;"}

Network Flexibility

We built Piggydrive using **Tailscale** for the private network connection — it's simple to set up and works great across different networks. But the key requirement is just that the two devices can maintain a direct connection. Any VPN, mesh network, or direct SSH tunnel will work. Tailscale is just the tool we chose for convenience.

{: style="text-align: justify;"}

Real-World Usage Patterns

Pattern 1: Find and Download Research Files

```
# 1. Find all files related to a project
piggydrive find "research" --max 100

# 2. Filter to the file types you need (e.g., PDFs)
# 3. Download each one to your local workspace
piggydrive pull "/Cloud/research/document.pdf" ~/work/
document.pdf
```

{: style="text-align: justify;"}

Pattern 2: Save Work to Cloud

```
# Generate a report locally
python generate_report.py > ~/work/report.md

# Push to cloud for access from other devices
piggydrive push ~/work/report.md "/Cloud/reports/report.md"

# Verify sync completed
piggydrive sync-status
```

{: style="text-align: justify;"}

Pattern 3: Quick Read Without Download

```
# Read a small file directly (downloads to temp, prints to
screen)
piggydrive cat "/Cloud/notes.md"
```

{: style="text-align: justify;"}

Why This Approach Works

No Authentication Fighting

The laptop's official client handles all authentication. Linux never touches authentication tokens. The organization's policies don't matter — the laptop is already trusted.

{: style="text-align: justify;"}

Works with Cloud-Only Files

Modern cloud storage uses "files on demand" — files exist as placeholders until opened. Piggydrive handles this transparently. Search finds placeholders instantly. Download triggers the actual file retrieval automatically.

{: style="text-align: justify;"}

Fast Search

Built-in search on Mac and Windows is extremely fast. Searching a large file tree takes under a second. Compare that to recursive directory traversal (which would timeout) or remote listing (which requires downloading everything).

{: style="text-align: justify;"}

Automation-Friendly

The CLI outputs structured data, has stable error codes, and is designed for automation. It works well in scripts and with agent workflows.

{: style="text-align: justify;"}

Limitations (Being Honest)

- **Requires a laptop bridge:** The daemon runs on macOS or Windows. You need a trusted machine already syncing cloud storage.
- **Private network dependency:** The bridge and CLI communicate over a private network. Both machines must be connected.
- **Bridge must be online:** If the laptop is asleep or offline, the CLI can't reach it.
- **Not a full cloud client:** You can't change cloud settings, manage sync rules, or do admin tasks. It's a filesystem proxy, not a replacement for the official client.

{: style="text-align: justify;"}

Call to Action

- **Was it fully vibe-coded?** Yes.
- **Does it solve my problem?** Absolutely.
- **Would I use it again?** I already do — daily.

Try it: The code is open source. If you're blocked by authentication restrictions, Piggydrive might be your workaround.

Contribute: Found a bug? Want a feature? The daemon and CLI are straightforward Python — fix it with your favorite tools.

{: style="text-align: justify;"}